

Security Audit

Shezmu 2nd (CDP)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	54
Conclusion	55
Our Methodology	56
Disclaimers	58
About Hashlock	59

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Shezmu team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Shezmu introduces a groundbreaking hybrid Collateralized Debt Position (CDP) platform that innovatively combines the capabilities of both NFTs and Yield-Bearing Tokens. The platform allows users to borrow against both NFTs and Yield-Bearing Tokens, providing unparalleled flexibility and liquidity in the digital asset space. In addition to the core CDP functionality, the project offers a suite of utilities designed to enhance user experience and asset value.

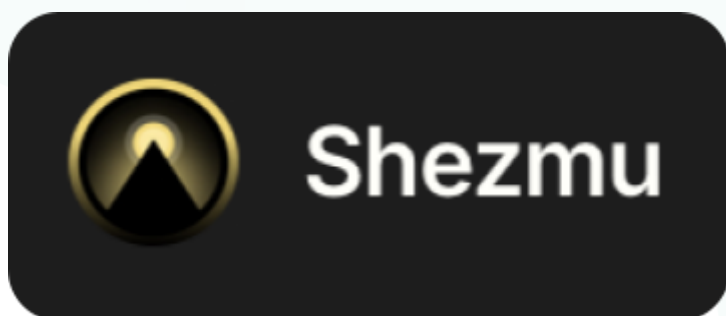
Project Name: Shezmu

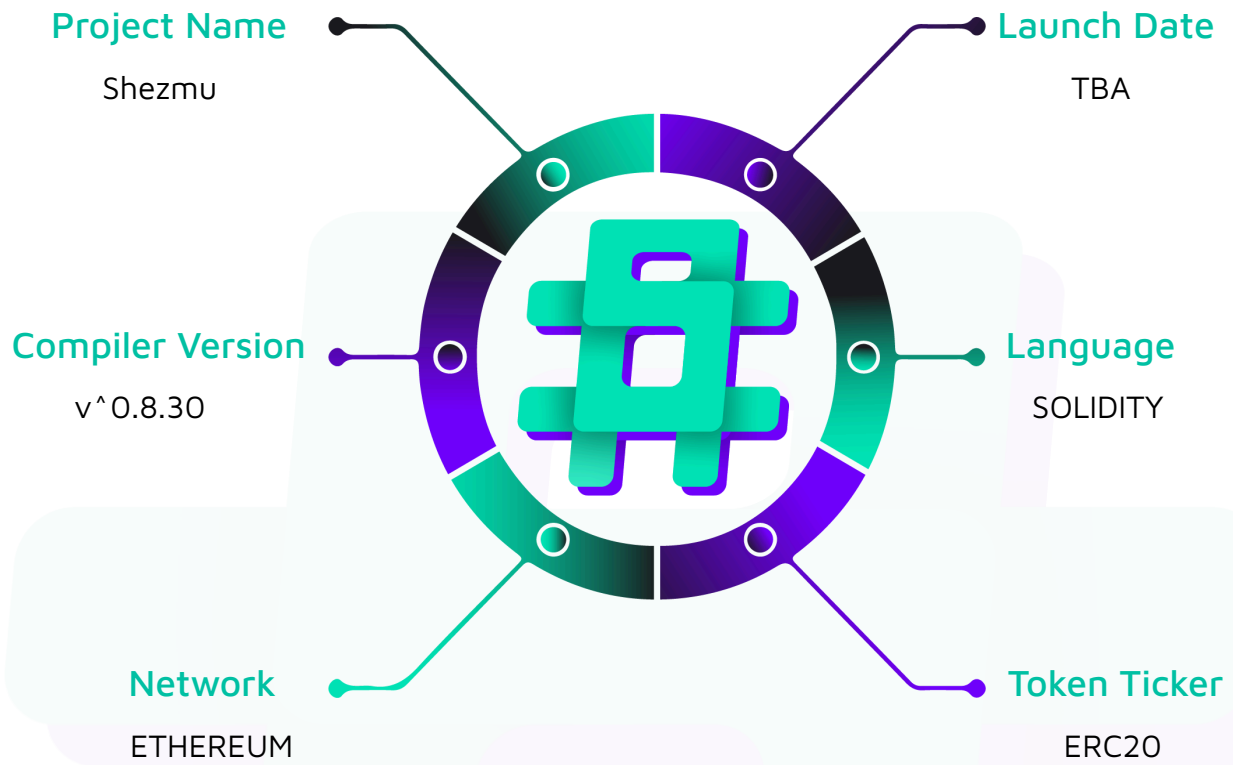
Project Type: Lending/Borrowing.

Compiler Version: ^0.8.30

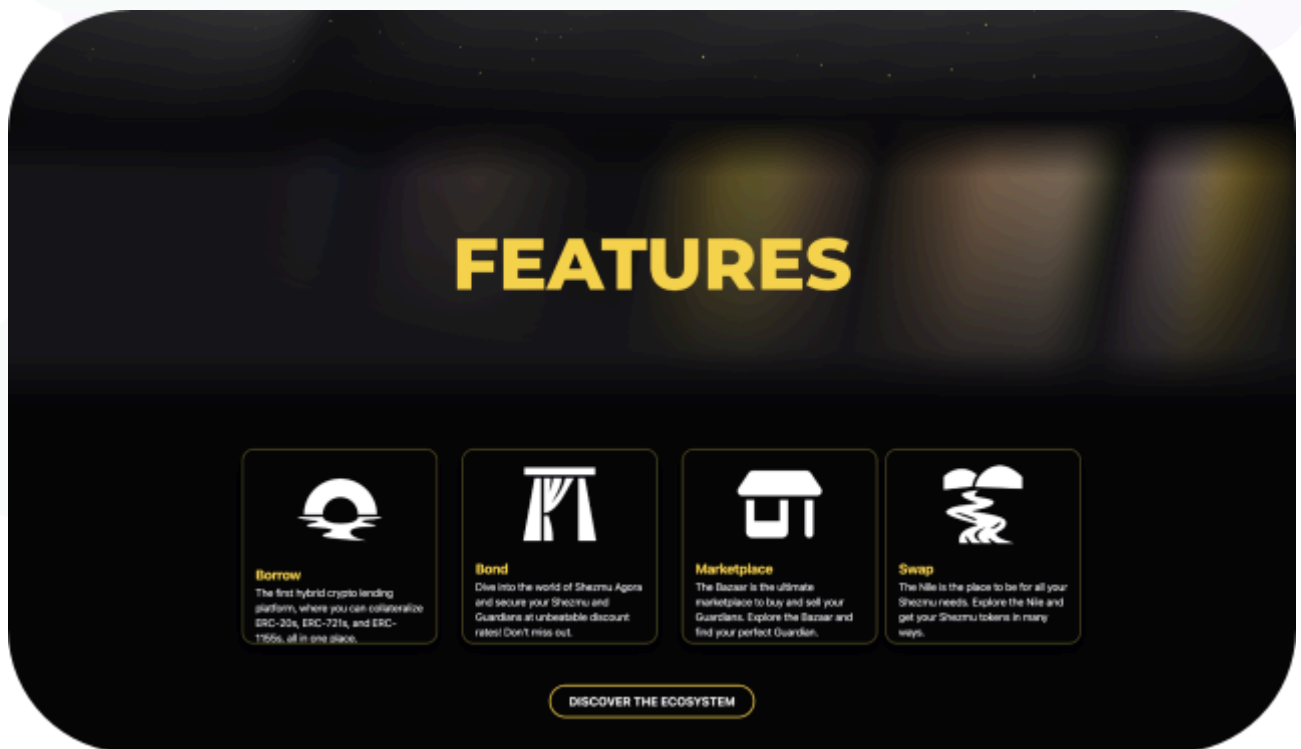
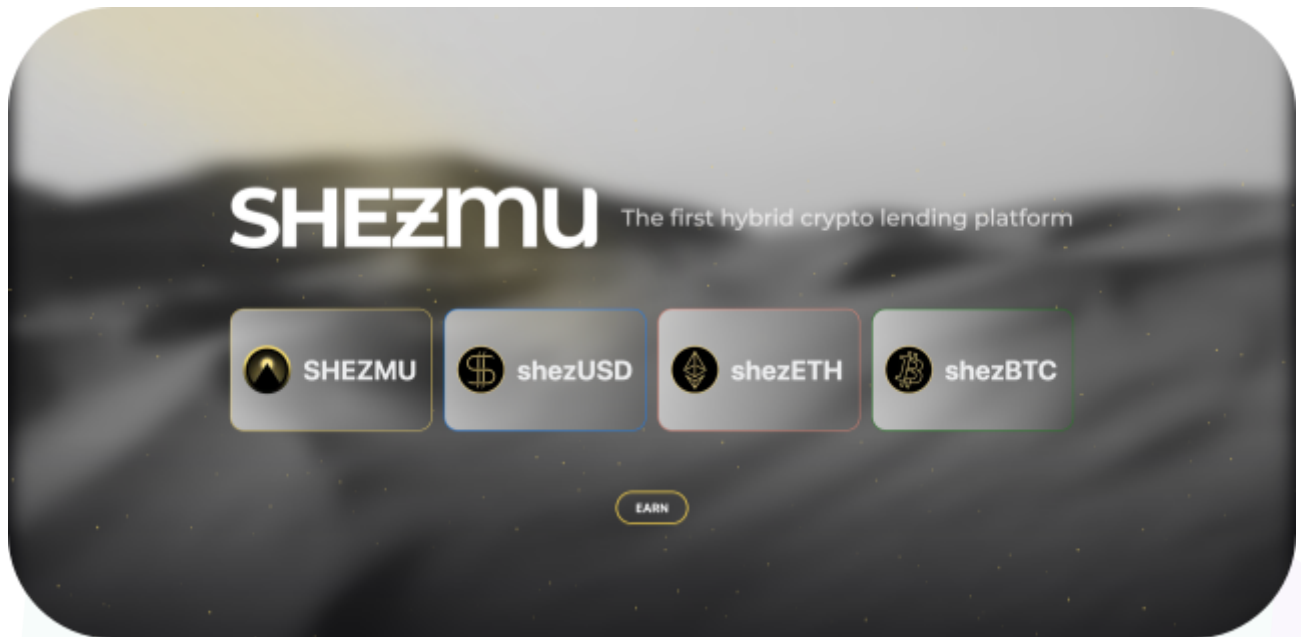
Website: www.shezmu.io

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Shezmu project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Shezmu Smart Contracts
Platform	Ethereum / Solidity
Audit Date	July, 2025
Contract 1	ERC20Vault.sol
Contract 1 MD5 Hash	d56dcfbf79eb79297ca56d8096471b46
Contract 2	InterestCollector.sol
Contract 2 MD5 Hash	0e9682efa0ad7374dac9ef74a3588d01
Contract 3	SoulBound.sol
Contract 3 MD5 Hash	21a35f784228c640c38f2b19b92cadb4
Contract 4	AaveStrategy.sol
Contract 4 MD5 Hash	4f40fe183140f7ae80195e4efd1313b5
Audited GitHub Commit Hash	9905f16491672392664d076e252c684b53df1333
Fix Review GitHub Commit Hash	8670ba6d501a1fec9fcac495851f3f643aec0843

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section, and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

6 Medium severity vulnerabilities

5 Low severity vulnerabilities

1 Gas Optimisation

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
ERC20Vault.sol - Open and manage leveraged lending positions with ERC20 collateral and borrowable tokens - Handle collateral deposits, withdrawals, and debt borrowing/repayment operations - Calculate position health factors and determine liquidation eligibility based on LTV ratios - Execute single and batch liquidations of undercollateralized positions with rewards and penalties - Integrate with external yield strategies for collateral optimization when users opt into interest accrual - Mint soul-bound NFT tokens for positions with associated fee structures - Collect and distribute interest payments through configurable interest collector mechanisms - Maintain user position tracking with collateral and loan balance accounting across multiple positions - Interface with price feed oracles for real-time collateral and loan token valuations - Provide leverage multipliers up to 10x with dynamic effective LTV ratio calculations - Handle emergency administrative functions, including asset recovery and parameter updates	Contract achieves this functionality.
InterestCollector.sol - Register lending vaults with configurable annual	Contract achieves this functionality.

interest rates and manage rate updates - Calculate and collect periodic interest from registered vaults - Accumulate collected interest for different loan tokens and provide treasury withdrawal functionality - Enforce collection timing requirements using configurable period blocks	
SoulBound.sol - Create non-transferable ERC721 tokens representing vault lending positions - Allow only the associated vault contract to mint and burn soul-bound tokens - Prevent all token transfers by overriding transferFrom and safeTransferFrom functions	Contract achieves this functionality.
AaveStrategy.sol - Deposit vault collateral into Aave v3 protocol - Withdraw collateral from Aave positions on behalf of the vault contract - Track collateral amounts per position ID and maintain total collateral accounting - Claim accumulated Aave rewards and forward them to the designated treasury address - Collect accrued interest from Aave deposits and automatically redeposit for compound yield - Enforce vault-only permissions for deposit/withdraw operations while allowing owner reward management - Support emergency token withdrawals and protocol address updates for contract maintenance	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Shezmu project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation; however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Shezmu project smart contract code in the form of GitHub access. As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] ERC20Vault#getPositionHealth - HF increases with increasing leverage.

Description

The `getPositionHealth()` function contains a logical error whereby the Health Factor increases with leverage, which contradicts the basic principles of risk management in DeFi protocols.

Vulnerability Details

In this line, the numerator increases by leverage, which ultimately shows a slow growth of the denominator relative to the numerator.

```
/// @notice Calculates the health factor of a position
/// @param positionId The ID of the position to check
/// @return The health factor (collateral value / debt value) * PRECISION
function getPositionHealth(
    uint256 positionId
) public view returns (uint256) {
    Position memory pos = positions[positionId];
    if (pos.debtAmount == 0) return type(uint256).max;

    uint256 collateralValue = getCollateralValue(pos.collateralAmount);
    uint256 debtValue = getLoanValue(pos.debtAmount);

=>    uint256 x = (collateralValue * pos.leverage * pos.effectiveLtvRatio) /
        100;
    uint256 y = (debtValue * (1000 - (1000 / (pos.leverage + 1)))) / 1000;

    return (x * PRECISION) / y;
}
```

Proof of Concept

The PoC shows that as leverage increases, HF increases.

```
function test_H_08_HealthFactor() public {
    vm.startPrank(user1);
    vault.setDoNotMint(true);
    uint256 collateralAmount = 1250 ether;
    uint256 debtAmount = (collateralAmount * INITIAL_LTV) / 100;
    uint256 leverage = 5;
    WETH.approve(address(vault), collateralAmount);

    assertEq(vault.ltvRatio(), 80);

    vault.openPosition(
        user1,
        address(WETH),
        collateralAmount,
        debtAmount,
        leverage
    );
    console.log(
        "Health Factor of first position without leverage == 5: ",
        vault.getPositionHealth(1)
    );

    vm.startPrank(user2);
    vault.setDoNotMint(true);
    uint256 collateralAmount_2 = 1250 ether;
    uint256 debtAmount_2 = (collateralAmount_2 * INITIAL_LTV) / 100;
    uint256 leverage_2 = 10;
    WETH.approve(address(vault), collateralAmount);

    vault.openPosition(
        user2,
        address(WETH),
        collateralAmount_2,
        debtAmount_2,
        leverage_2
    );
}
```

```

    console.log(
      "Health Factor of second position with leverage == 10: ",
      vault.getPositionHealth(2)
    );
  }
}

```

Test result:

```

Ran 1 test for test/ERC20VaultTest.t.sol:ERC20VaultTest
[PASS] test_H_08_HealthFactor() (gas: 786194)
Logs:
  Health Factor of first position without leverage == 5:    5995203836930455635
  Health Factor of second position with leverage == 10:    10989010989010989010

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.70s (562.90ms CPU time)

Ran 1 test suite in 11.99s (10.70s CPU time): 1 tests passed, 0 failed, 0 skipped (1
total tests)

---
cast --from-wei 5995203836930455635 && cast --from-wei 10989010989010989010
5.995203836930455635
10.989010989010989010

```

Impact

- High leverage positions cannot be liquidated
- Accumulation of bad debts in the protocol
- The riskiest positions appear to be the safest
- The protocol may incur significant losses

Recommendation

Correct the formula so that HF decreases as leverage increases.

Status

Resolved

[H-02] ERC20Vault - Missing storage update leaves position data unchanged

Description

The `_collectInterestIfAvailable` function loads a position into memory but modifies the memory variable instead of storage, causing interest to not be added to the position's debt amount.

Vulnerability Details

In the `_collectInterestIfAvailable` function, the position is loaded as a memory variable:

```
function _collectInterestIfAvailable(
    uint256 positionId
) internal returns (uint256) {
    if (
        address(interestCollector) == address(0) ||
        !interestCollectionEnabled
    ) return 0;

    Position memory pos = positions[positionId];

    if (pos.debtAmount == 0) return 0;

    try
        interestCollector.collectInterest(
            address(this),
            address(loanToken),
            positionId,
            pos.debtAmount
        )
    returns (uint256 interestAmount) {
```

```
    loanToken.mint(address(interestCollector), interestAmount);

    totalDebt += interestAmount;

    pos.debtAmount += interestAmount;

    loanBalances[pos.owner] += interestAmount;

    return interestAmount;
} catch {
    return 0;
}
```

The line `pos.debtAmount += interestAmount` only modifies the memory copy of the position, not the actual storage. The storage position remains unchanged.

Impact

Interest is minted but not added to the position's debt. Protocol accounting becomes inconsistent: `totalDebt` increases, but individual position debts don't.

Recommendation

Save the modified variable back to storage at the end of the function.

Status

Resolved

[H-03] ERC20Vault#liquidatePosition/batchLiquidate - DoS in liquidation functions.

Description

When liquidation functions such as `liquidatePosition` and `batchLiquidate` are called, the `loanToken` is burned. If the user transfers this token to another address, these functions will revert, thereby causing a DoS.

Vulnerability Details

The protocol attempts to burn loan tokens from the position owner's balance upon liquidation, but this operation will be canceled if:

- 1) The position owner has already spent their loan tokens.
- 2) The position owner has insufficient balance due to transfers or other operations.

```
function liquidatePosition(uint256 positionId) external nonReentrant {
    //...code
    uint256 collateralAmount = position.collateralAmount;
    uint256 debtAmount = position.debtAmount;
    //...code

    loanToken.burn(positionOwner, debtAmount);
    emit PositionLiquidated(positionId, msg.sender, reward, debtAmount);
}

function batchLiquidate(
    uint256[] calldata positionIds
) external nonReentrant {
    //...code
    for (uint256 i = 0; i < positionIds.length; i++) {
        uint256 positionId = positionIds[i];
        Position storage position = positions[positionId];
        address positionOwner = position.owner;
```

```

    if (positionOwner == address(0)) continue;
    if (!isLiquidatable(positionId)) continue;

    _collectInterestIfAvailable(positionId);

    uint256 collateralAmount = position.collateralAmount;
    uint256 debtAmount = position.debtAmount;
    //...code
    loanToken.burn(positionOwner, debtAmount);

    _deletePosition(positionId, positionOwner);
}

//...code
}

emit BatchPositionsLiquidated(positionIds, msg.sender, totalReward);
}

```

Proof of Concept

PoC demonstrates that the user does not pay a fee and mints SoulBound tokens.

```

function test_H07_LiquidatePositionRevert() public {
    vm.startPrank(user1);
    uint256 collateralAmount = 1000 ether;
    uint256 debtAmount = (collateralAmount * INITIAL_LTV) / 100;
    uint256 leverage = 1;
    WETH.approve(address(vault), collateralAmount);

    vault.openPosition(
        user1,
        address(WETH),

```

```

        collateralAmount,
        debtAmount,
        leverage
    );
    //Change the price so that the position becomes liquidatable.
    wethPriceFeed.setPrice(50 * 10 ** 6);

    assertTrue(vault.isLiquidatable(1));

    //Transfer the debt to another user.
    assertEquals(vault.loanToken().balanceOf(user1), debtAmount);
    vault.loanToken().transfer(user2, debtAmount);
    assertEquals(vault.loanToken().balanceOf(user2), debtAmount);
    assertEquals(vault.loanToken().balanceOf(user1), 0);

    vm.expectRevert();
    vault.liquidatePosition(1);

    vm.stopPrank();
}

```

Test result:

```

Ran 1 test for test/ERC20VaultTest.t.sol:ERC20VaultTest
[PASS] test_H_07_LiquidatePositionRevert() (gas: 528714)

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.91s (406.68ms CPU time)

Ran 1 test suite in 12.18s (10.91s CPU time): 1 tests passed, 0 failed, 0 skipped (1
total tests)

```

Impact

Undercollateralized positions cannot be liquidated, leading to bad debt accumulation



Hashlock Pty Ltd

Upon liquidation, the user suffers a double loss. Part of their collateral is taken away, and their `loanToken` is burned.

Recommendation

Consider getting rid of `loanToken` burning.

Status

Resolved

Medium

[M-01] ERC20Vault.sol - _getPrice lacks freshness checks

Description

The `_getPrice` function in `ERC20Vault.sol` calls `latestRoundData()` but doesn't verify `updatedAt` or `answeredInRound` parameters, allowing stale or manipulated prices to be used for operations like liquidations and borrowing calculations.

Impact

Stale or incomplete oracle data could lead to incorrect liquidations or allow over-borrowing beyond safe limits.

Recommendation

Add staleness and round validation checks.

Status

Resolved

[M-02] ERC20Vault.sol - collateralToken will fail due to multiple reasons if its USDT or equivalent

Description

When `collateralToken` is a nonstandard ERC-20 (e.g. USDT),

- `transferFrom(...)` doesn't return a bool and will revert when called via `IERC20.transferFrom`.
- `approve(spender, amount)` will revert if the existing allowance is nonzero.

Impact

This effectively makes the contract non-functional if USDT is used as collateral.

Recommendation

Replace direct calls with OpenZeppelin's SafeERC20 wrappers—use `SafeERC20.safeTransferFrom(...)` and `safeIncreaseAllowance/safeDecreaseAllowance` (or reset to zero before approve), ensuring compatibility with tokens like USDT.

Status

Resolved

[M-03] ERC20Vault.sol/InterestCollector.sol - Loss of accuracy during division

Description

The functions `ERC20Vault#_estimateTotalDebt` and `InterestCollector#calculateInterestDue` contain vulnerabilities related to loss of precision during division. In particular, because of this, the function `_estimateTotalDebt` allows you to take more credit tokens than you are entitled to, and in the function `calculateInterestDue`, the owner receives fewer tokens.

Vulnerability Details

Let's look at each function individually.

In the function `_estimateTotalDebt`, the following line causes a major problem

```
uint256 halfDifference = differenceTo100 / 2;
```

In this case, accuracy is lost if LTV is an odd number.

```
function _estimateTotalDebt(
    address user,
    uint256 collateralAmount,
    uint256 leverage,
    bool open
)
    public
    view
    returns (
        uint256 fee,
        uint256 adjustedCollateral,
        uint256 debtAmount,
        uint256 effectiveLtvRatio
    )
{
    uint256 baseLtvRatio = ltvRatio;
    effectiveLtvRatio = baseLtvRatio;
    adjustedCollateral = collateralAmount;
```

```

if (open && !doNotMint[user]) {
    fee = (collateralAmount * soulBoundFeePercent) / 100;
    adjustedCollateral = collateralAmount - fee;

    uint256 differenceTo100 = 100 - baseLtvRatio;
=>    uint256 halfDifference = differenceTo100 / 2;
    uint256 newLtvBeforeFee = 100 - halfDifference;
    effectiveLtvRatio = newLtvBeforeFee - soulBoundFeePercent;
}

uint256 collateralValue = getCollateralValue(adjustedCollateral);
uint256 loanValueNeeded = collateralValue * leverage;
uint256 loanValue = (loanValueNeeded * effectiveLtvRatio) / 100;

uint256 loanPrice = getLoanValue(1 ether);

debtAmount = (loanValue * PRECISION) / loanPrice;
}

```

If LTV is equal to 75, then newLtvBeforeFee will be equal to 88, but it should be equal to 87.5.

In the function calculateInterestDue, the problem with loss of precision occurs because division takes place before multiplication.

```

function calculateInterestDue(
    address vault,
    uint256 positionId,
    uint256 debtAmount
) public view returns (uint256) {
    if (vaultInterestRates[vault] == 0) return 0;
    if (lastCollectionBlock[vault][positionId] == 0) return 0;
    if (debtAmount == 0) return 0;
}

```

```

uint256 currentBlock = block.number;
if (currentBlock <= lastCollectionBlock[vault][positionId]) return 0;

uint256 blocksPassed = currentBlock -
    lastCollectionBlock[vault][positionId];
=> uint256 periodsPassed = blocksPassed / periodBlocks;

if (periodsPassed == 0) return 0;

uint256 annualInterest = debtAmount * vaultInterestRates[vault];
uint256 periodInterest = (annualInterest * periodShare) /
    (10000 * PRECISION);

=> return periodInterest * periodsPassed;
}

```

Proof of Concept

The following PoC compares the results of the original function and the modified one, with greater accuracy, where 1000 is 100%.

Add the following function to the main contract and change the visibility of the original function to public:

```

function _estimateTotalDebtModified(
    address user,
    uint256 collateralAmount,
    uint256 leverage,
    bool open
)
    public
    view
    returns (
        uint256 fee,
        uint256 adjustedCollateral,
        uint256 debtAmount,

```

```

        uint256 effectiveLtvRatio
    )
    {
        uint256 baseLtvRatio = ltvRatio * 10; // Increasing accuracy - 75% now 750
        effectiveLtvRatio = baseLtvRatio;
        adjustedCollateral = collateralAmount;

        if (open && !doNotMint[user]) {
            fee = (collateralAmount * 20) / 1000; // Increased by the fee amount - 2% now
20
            adjustedCollateral = collateralAmount - fee;

            uint256 differenceTo100 = 1000 - baseLtvRatio;
            uint256 halfDifference = differenceTo100 / 2;
            uint256 newLtvBeforeFee = 1000 - halfDifference;

            effectiveLtvRatio = newLtvBeforeFee - 20; // Increased by the fee amount - 2%
now 20
        }
        uint256 collateralValue = getCollateralValue(adjustedCollateral);

        uint256 loanValueNeeded = collateralValue * leverage;
        uint256 loanValue = (loanValueNeeded * effectiveLtvRatio) / 1000;
        uint256 loanPrice = getLoanValue(1 ether);
        debtAmount = (loanValue * PRECISION) / loanPrice;
    }

```

Run the following test:

```

function test_M_03_estimateTotalDebt() public {
    assertEq(vault.ltvRatio(), 75);
    assertEq(vault.soulBoundFeePercent(), 2);
    (
        uint256 fee,
        uint256 adjustedCollateral,
        uint256 debtAmount,
        uint256 effectiveLtvRatio
    ) = vault._estimateTotalDebt(user1, 100 ether, 1, true);
}

```

```

    (
        uint256 feeModified,
        uint256 adjustedCollateralModified,
        uint256 debtAmountModified,
        uint256 effectiveLtvRatioModified
    ) = vault._estimateTotalDebtModified(user1, 100 ether, 1, true);

    assertEq(feeModified, fee);
    assertEq(adjustedCollateralModified, adjustedCollateral);
    assertEq(debtAmountModified, debtAmount, "Not Equal");
}

```

Test result

```

Ran 1 test for test/ERC20VaultTest.t.sol:ERC20VaultTest
[FAIL:      Not      Equal:      83790000000000000000      !=      84280000000000000000]
test_M_03_estimateTotalDebt() (gas: 60427)

Suite result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 13.95s (4.01ms CPU time)

Ran 1 test suite in 15.40s (13.95s CPU time): 0 tests passed, 1 failed, 0 skipped (1
total tests)

Failing tests:
Encountered 1 failing test in test/ERC20VaultTest.t.sol:ERC20VaultTest
[FAIL:      Not      Equal:      83790000000000000000      !=      84280000000000000000]
test_M_03_estimateTotalDebt() (gas: 60427)

Encountered a total of 1 failing tests, 0 tests succeeded

cast --from-wei 83790000000000000000
83.790000000000000000

cast --from-wei 84280000000000000000
84.280000000000000000

```

As you can see, due to the division problem, users are allowed to take more loan tokens equal to almost 0.5 ether. This is a small difference if it is a stablecoin, but in the case of ETH, it is a big difference.



The following PoC demonstrates an issue with division in the `calculateInterestDue` function.

Add the following function to the `InterestCollector` contract

```
function calculateInterestDueModified(
    address vault,
    uint256 positionId,
    uint256 debtAmount
) public view returns (uint256) {
    if (vaultInterestRates[vault] == 0) return 0;
    if (lastCollectionBlock[vault][positionId] == 0) return 0;
    if (debtAmount == 0) return 0;

    uint256 currentBlock = block.number;
    if (currentBlock <= lastCollectionBlock[vault][positionId]) return 0;

    uint256 blocksPassed = currentBlock -
        lastCollectionBlock[vault][positionId];
    uint256 periodsPassed = blocksPassed / periodBlocks;

    if (periodsPassed == 0) return 0;

    uint256 annualInterest = debtAmount * vaultInterestRates[vault];
    uint256 periodInterest = (annualInterest * periodShare) /
        (10000 * PRECISION);

    return (periodInterest * blocksPassed) / periodBlocks;
}
```

Add and run the following test

```
function test_M_03_calculateInterestDue() public {
    vm.startPrank(user1);
    uint256 collateralAmount = 1000 ether;
    uint256 debtAmount = (collateralAmount * INITIAL_LTV) / 100;
    uint256 leverage = 1;
    WETH.approve(address(vault), collateralAmount * 2);

    vault.openPosition(
```

```

        user1,
        address(WETH),
        collateralAmount,
        debtAmount,
        leverage
    );

    vm.roll(block.number + 400);

    uint256 interestDue = interestCollector.calculateInterestDue(
        address(vault),
        1,
        debtAmount
    );
    uint256 interestDueModified = interestCollector
        .calculateInterestDueModified(address(vault), 1, debtAmount);
    assertEq(interestDueModified, interestDue);
}

```

Test result:

```

Ran 1 test for test/ERC20VaultTest.t.sol:ERC20VaultTest
[FAIL:      assertion      failed:      5739649498737249      !=      4304737124052937]
test_M_03_calculateInterestDue() (gas: 564993)

Suite result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 9.97s (390.46ms CPU
time)

Ran 1 test suite in 11.24s (9.97s CPU time): 0 tests passed, 1 failed, 0 skipped (1 total
tests)

Failing tests:
Encountered 1 failing test in test/ERC20VaultTest.t.sol:ERC20VaultTest
[FAIL:      assertion      failed:      5739649498737249      !=      4304737124052937]
test_M_03_calculateInterestDue() (gas: 564993)

Encountered a total of 1 failing tests, 0 tests succeeded

cast --from-wei 5739649498737249

```

```
0.005739649498737249
```

```
cast --from-wei 4304737124052937
```

```
0.004304737124052937
```

Impact

When accuracy is lost, users will be issued more loan tokens, which may result in users' positions being subject to liquidation more quickly. In the case of InterestCollector, the owner will receive fewer tokens.

Recommendation

Consider changing the precision from 100 to 1000. And perform multiplication before division.

Status

Resolved

[M-04] ERC20Vault.sol - SoulBound fee payment doesn't provide promised borrowing benefits for subsequent borrows

Description

The `ERC20Vault` allows users to pay a 2% fee to mint a `SoulBound` NFT, which promises better borrowing terms. However, the improved terms only apply to the initial borrow, not subsequent borrows from the same position.

Vulnerability Details

The protocol offers two ways to open a position:

1. Without `SoulBound` (free): Users get the default LTV ratio (e.g., 50%)
2. With `SoulBound` (2% fee): Users pay 2% of their collateral for an improved LTV ratio

When opening a position with `SoulBound`:

```
function openPosition(...) external {
    // If user hasn't opted out of SoulBound
    if (!doNotMint[owner]) {
        // Calculate improved LTV
        uint256 baseLtvRatio = ltvRatio; // e.g., 50%
        uint256 differenceTo100 = 100 - baseLtvRatio; // 50
        uint256 halfDifference = differenceTo100 / 2; // 25
        uint256 newLtvBeforeFee = 100 - halfDifference; // 75
        effectiveLtvRatio = newLtvBeforeFee - soulBoundFeePercent; // 73%

        // User pays 2% fee
        fee = (collateralAmount * soulBoundFeePercent) / 100;
        adjustedCollateral = collateralAmount - fee;
    }
}
```

The position stores this improved `effectiveLtvRatio` (73% vs 50% default). However, when users later call `borrow()` to take additional loans:

```
function _borrow(...) internal {
    // This recalculates using default LTV because open=false
    (, , uint256 maxLoan, ) = _estimateTotalDebt(
        onBehalfOf,
        position.collateralAmount,
        position.leverage,
        false // This makes it use the default 50% LTV!
    );
}
```

The `_estimateTotalDebt` function with `open=false` ignores the stored `effectiveLtvRatio` and uses the default vault LTV:

```
function _estimateTotalDebt(..., bool open) internal {
    if (open && !doNotMint[user]) {
        // Calculate improved LTV
    } else {
        effectiveLtvRatio = baseLtvRatio; // Uses default!
    }
}
```

Example:

- User deposits 100 ETH, pays 2 ETH fee
- With 73% LTV on 98 ETH collateral, you can initially borrow up to 71.54 ETH worth
- Later, when trying to borrow more, limited to 50% LTV = 49 ETH worth of total
- The user paid 2 ETH to access a 71.54 ETH credit line, but can only ever use 49 ETH

Impact

Users pay a 2% fee for improved LTV but only benefit from it during initial position opening, losing access to ~23% additional borrowing capacity for all subsequent borrows.

Recommendation

Use the position's stored `effectiveLtvRatio` for all borrowing calculations instead of recalculating based on the `open` flag.

Status

Resolved

[M-05] ERC20Vault.sol#_addCollateral - Adding collateral to two users at once.

Description

The `_addCollateral` function adds collateral for `onBehalfOf`, but due to the lack of verification that `onBehalfOf` is the owner of this position, a vulnerability is created that allows increasing the `collateralAmount` in the position structure, as well as mapping `collateralBalances` for the `onBehalfOf` address.

```
function _addCollateral(
    address account,
    address onBehalfOf,
    uint256 positionId,
    uint256 amount
) internal {
    if (amount == 0) revert ZeroCollateralAmount();
    if (!collateralToken.transferFrom(account, address(this), amount)) {
        revert CollateralTransferFailed();
    }
    if (
        address(strategy) != address(0) &&
        positions[positionId].interestOptOut
    ) {
        collateralToken.approve(address(strategy), amount);
        strategy.deposit(positionId, amount);
    }

    positions[positionId].collateralAmount += amount;
    collateralBalances[onBehalfOf] += amount;
    emit CollateralAdded(positionId, amount);
}
```

Vulnerability Details

Due to the lack of verification, the deposit in the mapping may be less than the value in the structure. This can lead to an underflow error during position liquidation, as a larger value will be subtracted from the mapping.

```
function liquidatePosition(uint256 positionId) external nonReentrant {
    Position storage position = positions[positionId];
    address positionOwner = position.owner;
    => uint256 collateralAmount = position.collateralAmount;
    uint256 debtAmount = position.debtAmount;
    bool posInterestOptOut = position.interestOptOut;

    if (positionOwner == address(0)) revert InvalidPosition();
    if (!isLiquidatable(positionId)) revert PositionNotLiquidatable();

    _collectInterestIfAvailable(positionId);

    uint256 reward = (collateralAmount * liquidatorReward) / 100;
    uint256 penalty = (collateralAmount * penaltyRate) / 100;
    uint256 remainingCollateral = collateralAmount - reward - penalty;

    if (address(strategy) != address(0) && posInterestOptOut) {
        strategy.withdraw(positionId, collateralAmount);
    }

    if (!collateralToken.transfer(treasury, penalty)) {
        revert LiquidationFailed();
    }
    if (!collateralToken.transfer(msg.sender, reward)) {
        revert LiquidationFailed();
    }
    if (remainingCollateral > 0) {
        if (!collateralToken.transfer(positionOwner, remainingCollateral)) {
            revert LiquidationFailed();
        }
    }
}

=> collateralBalances[positionOwner] -= collateralAmount;
```

```
loanBalances[positionOwner] -= debtAmount;

totalDebt -= debtAmount;

_deletePosition(positionId, positionOwner);

loanToken.burn(positionOwner, debtAmount);
emit PositionLiquidated(positionId, msg.sender, reward, debtAmount);
}
```

Impact

Accumulation of bad debts.

Recommendation

Add a check to ensure that onBehalfOf is the owner of the position.

Status

Resolved

[M-06] ERC20Vault#getPositionHealth - Incorrect calculation of the health factor

Description

The `getPositionHealth` function in the `ERC20Vault` contract contains a flaw in the health factor calculation formula, which significantly overestimates the health of a position, making positions appear safer than they are and preventing liquidation.

Vulnerability Details

From the comment on this function, HF is calculated as the ratio of collateral to debt.

```

    /// @notice Calculates the health factor of a position
    /// @param positionId The ID of the position to check
=> /// @return The health factor (collateral value / debt value) * PRECISION
    function getPositionHealth(
        uint256 positionId
    ) public view returns (uint256) {
        Position memory pos = positions[positionId];
        if (pos.debtAmount == 0) return type(uint256).max;

        uint256 collateralValue = getCollateralValue(pos.collateralAmount);
        uint256 debtValue = getLoanValue(pos.debtAmount);

        uint256 x = (collateralValue * pos.leverage * pos.effectiveLtvRatio) /
            100;
        uint256 y = (debtValue * (1000 - (1000 / (pos.leverage + 1)))) / 1000;

        return (x * PRECISION) / y;
    }

```

Proof of Concept

This PoC compares two HF calculations: one using the main function and one using the formula described in the NatSpec comment.

```

function test_H_07_HealthFactor() public {
    vm.startPrank(user1);
    vault.setDoNotMint(true);

```

```

uint256 collateralAmount = 1250 ether;
uint256 debtAmount = (collateralAmount * INITIAL_LTV) / 100;
uint256 leverage = 1;
WETH.approve(address(vault), collateralAmount);

assertEq(vault.ltvRatio(), 80);

vault.openPosition(
    user1,
    address(WETH),
    collateralAmount,
    debtAmount,
    leverage
);

uint256 hfModified = (vault.getCollateralValue(collateralAmount) *
    vault.PRECISION()) / (vault.getLoanValue(debtAmount));

assertEq(vault.getPositionHealth(1), hfModified);
}

```

Test result:

```

Ran 1 test for test/ERC20VaultTest.t.sol:ERC20VaultTest
[FAIL: assertion failed: 2000000000000000000 != 1250000000000000000]
test_H_07_HealthFactor() (gas: 478106)

Suite result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 11.14s (287.30ms CPU
time)

Ran 1 test suite in 12.52s (11.14s CPU time): 0 tests passed, 1 failed, 0 skipped (1
total tests)

Failing tests:
Encountered 1 failing test in test/ERC20VaultTest.t.sol:ERC20VaultTest
[FAIL: assertion failed: 2000000000000000000 != 1250000000000000000]
test_H_07_HealthFactor() (gas: 478106)

Encountered a total of 1 failing tests, 0 tests succeeded

```

```
---  
cast --from-wei 2000000000000000000  
2.000000000000000000  
  
cast --from-wei 1250000000000000000  
1.250000000000000000
```

Impact

This function is the basis of the protocol, as it is used to check positions for liquidation. Since this function overestimates HF, it will result in all positions with bad collateral being considered sufficiently collateralized, which will lead to the protocol becoming insolvent.

Recommendation

Correct the Health Factor formula to properly calculate the ratio of collateral value to debt value, considering leverage

Status

Resolved

Low

[L-01] InterestCollector - Blocks per year hardcoded may become inaccurate multichain or on hardforks

Description

The `blocksPerYear` variable in `InterestCollector.sol` is hardcoded to `7160 * 365` and cannot be updated by the owner, making interest calculations inaccurate when deployed on different blockchains or after hardforks that change block times. This value assumes approximately 12-second block times (7160 blocks per day), which may be accurate for Ethereum mainnet but becomes incorrect on other chains:

- Polygon: ~2 second block times
- BSC: ~3 second block times
- Arbitrum: Variable block times

Additionally, Ethereum's block times have changed over time (especially after the Merge to Proof-of-Stake) and could change again in future hardforks.

Recommendation

Add a function to allow the owner to update `blocksPerYear`

Status

Acknowledged

[L-02] ERC20Vault - Position owner can liquidate their own position

Description

The `liquidatePosition` and `batchLiquidate` functions in `ERC20Vault.sol` do not prevent position owners from liquidating their own positions. This allows position owners to receive the liquidator reward when their position becomes liquidatable, potentially gaming the liquidation mechanism. This could be used to minimize losses during liquidation by capturing the liquidator reward.

Recommendation

Add a check to prevent self-liquidation if `msg.sender != positionOwner`.

Status

Acknowledged

[L-03] ERC20Vault#getCollateralValue/getLoanValue - If the stablecoin price is below \$1, zero will be returned.

Description

The functions `getCollateralValue` and `getLoanValue` calculate tokens in USD. However, if a stablecoin is used as a collateral token or credit token, and the price falls below 1 USD, these functions will return 0.

Vulnerability Details

If the oracle returns an asset price that is less than \$1, then when these functions are called with the argument 1, they will return 0 due to rounding issues.

Proof of Concept

```
function test_L_04_Price() public {  
    wethPriceFeed.setPrice(0.999 * 10 ** 6);  
    shezUSDPriceFeed.setPrice(0.999 * 10 ** 6);  
    assertEq(vault.getCollateralValue(1), 0);  
    assertEq(vault.getLoanValue(1), 0);  
}
```

Test result:

```
Ran 1 test for test/ERC20VaultTest.t.sol:ERC20VaultTest  
[PASS] test_L_04_Price() (gas: 48387)  
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.25s (2.37ms CPU time)  
  
Ran 1 test suite in 11.62s (10.25s CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
```

Recommendation

Add a check that the price is not equal to 0.

Status

Acknowledged

[L-04] Contracts - No check for 0 value

Description

Many functions do not check whether the argument passed is equal to 0, which could lead to unexpected behaviour in future.

Recommendation

Consider adding checks for arguments with a value of 0, not only in functions but also in constructors.

Status

Acknowledged

[L-05] SoulBound - Missing override for 3-arg `safeTransferFrom`

Description

The ERC721 interface includes two overloads of `safeTransferFrom`: one taking three arguments (`address,address,uint256`) and one taking four. The contract only overrides the four-argument variant, leaving the three-argument version still callable

However, while a separate PoC worked as a confirmation of this finding, PoC on the audited repo did not confirm the issue. Considering the ambiguity in results, it might still be worth patching.

Proof of Concept

A PoC was developed to demonstrate adequate contract behavior

```
(defun mint:bool (to:string amount:decimal)

  @doc "Mints amount of token to an account"

  (with-capability (SUPPLY_MANAGER)

    true

  )

  (with-capability (CREDIT to)

    (credit to (at 'guard (details to)) amount)

    (emit-event (MINT amount to))

  )

  (with-capability (SUPPLY)

    (update-supply amount)

  )

)

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.13;
```

```

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";

contract POCNFT is ERC721 {

    constructor() ERC721("POC", "POC") {

        _mint(msg.sender, 1); // mint token ID 1 to deployer

    }

    // override the 4-arg safeTransferFrom so it always reverts

    function safeTransferFrom(

        address from,

        address to,

        uint256 tokenId,

        bytes memory data

    ) public virtual override {

        revert("safeTransfer disabled");

    }

}

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.13;

import "forge-std/Test.sol";

import "../src/POCNFT.sol";

contract POCNFTTest is Test {

    POCNFT nft;

    address owner = address(this);

```

```

address other = address(0xBEEF);

function setUp() public {
    nft = new POCNFT();
    // token #1 is owned by address(this) from constructor
}

function testSafeTransfer4ArgsReverts() public {
    vm.expectRevert("safeTransfer disabled");
    nft.safeTransferFrom(owner, other, 1, "0x00");
}

function testTransferFrom3ArgsSucceeds() public {
    // before: owner owns #1
    assertEq(nft.ownerOf(1), owner);
    nft.transferFrom(owner, other, 1);
    assertEq(nft.ownerOf(1), other);
}
}

```

Impact

SouldBound logic can be bypassed.

Recommendation

Add the following override to `SoulBound.sol` to revert to the three-argument variant as well.

Status

Acknowledged

Gas

[G-01] ERC20Vault - Using immutable variables saves gas when calling functions.

Description

This contract uses variables that are assigned values in the constructor and remain unchanged. Examples include `collateralToken`, `loanToken`, `liquidationThreshold`, `liquidatorReward`, and `penaltyRate`. Such variables can be marked as `immutable` to save gas. While you will spend more gas when deploying this protocol as the bytecode will increase, you will reduce gas consumption when calling functions where these variables are used. This will offset the gas spent during deployment in the future.

Proof of Concept

```
function setUp() public {
    //...code
    uint256 gasBefore = gasleft();
    vault = new ERC20Vault(
        address(WETH),
        address(shezUSD),
        INITIAL_LTV,
        LIQUIDATION_THRESHOLD,
        LIQUIDATOR_REWARD,
        address(wethPriceFeed),
        address(shezUSDPriceFeed),
        treasury,
        address(strategy)
    );
    uint256 gasAfter = gasleft();
    console.log("Gas for deployment", gasBefore - gasAfter);
    //...code
}

function test_Deploy() public {
    vm.startPrank(user1);
```



```

uint256 collateralAmount = 1000 ether;
uint256 debtAmount = (collateralAmount * INITIAL_LTV) / 100;
uint256 repayAmount = 300 ether;

WETH.approve(address(vault), collateralAmount);
uint256 gasBefore = gasleft();
vault.openPosition(
    user1,
    address(WETH),
    collateralAmount,
    debtAmount,
    1
);
uint256 gasAfter = gasleft();
console.log("Gas for open position", gasBefore - gasAfter);
}

```

Test results using immutable variables and without

```

// Without Immutable
Ran 1 test for test/ERC20VaultTest.t.sol:ERC20VaultTest
[PASS] test_Deploy() (gas: 526811)
Logs:
  Gas for deployment 3589178
  Gas for open position 506660

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.28s (407.20ms CPU time)

Ran 1 test suite in 11.56s (10.28s CPU time): 1 tests passed, 0 failed, 0 skipped (1
total tests)

---

// With Immutable
Ran 1 test for test/ERC20VaultTest.t.sol:ERC20VaultTest
[PASS] test_Deploy() (gas: 522486)
Logs:

```

```
Gas for deployment 3619903
Gas for open position 502335

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 37.19s (2.26s CPU time)

Ran 1 test suite in 38.49s (37.19s CPU time): 1 tests passed, 0 failed, 0 skipped (1
total tests)
```

The test results show that using `immutable` variables increases the gas per deployment but decreases the gas per function call.

Recommendation

Consider changing variables to immutable

Status

Resolved

QA

[Q-01] Soulbound.sol - Missing override and revert to prevent approvals on non-transferable token

Description

Despite the token being non-transferrable, it still has methods inherited from ERC721 like `approve()` or `setApprovalForAll`.

These are not needed if the tokens are not transferable.

Recommendation

Override those methods to revert when called to prevent approvals on non-transferable tokens.

Status

Resolved

[Q-02] Multiple contracts - Missing event emissions

Description

There are multiple key state changing functions that do not emit events. This decreases on-chain transparency. For instance, the following functions exhibit that behavior:

- `InterestCollector::setPeriodBlocks`
- `ERC20Vault::updatePriceFeeds / updateLtvRatio / emergencyWithdraw`
- `AaveStrategy::setUserUseReserveAsCollateral`

Recommendation

Add event emission to all key state changing functions.

Status

Resolved

[Q-03] ERC20Vault#_deletePosition - TYPO

Description

There is a typo in NatSpec's comment.

```
/// @notice Internal function to burn souldbound token if exists
/// @param positionId The ID of the position to burn token for
function _deletePosition(uint256 positionId, address owner) internal {
    ///...code
}
```

Recommendation

Change souldbound to soulbound

Status

Resolved

[Q-04] Contracts - Update status before transfer.

Description

Even though most functions contain the nonReentrant modifier, the functions do not follow best practices, which require updating the state first and then transferring tokens.

For example:

```
function withdrawInterest(address token) external onlyOwner nonReentrant {
    uint256 amount = collectedInterest[token];
    if (amount == 0) revert NoInterestToCollect();

    bool success = IERC20(token).transfer(treasury, amount);
    if (!success) revert TransferFailed();

    collectedInterest[token] = 0;

    emit InterestWithdrawn(token, amount, treasury);
}
```

```
}
```

Recommendation

Consider updating the status first and then transfer the tokens.

Status

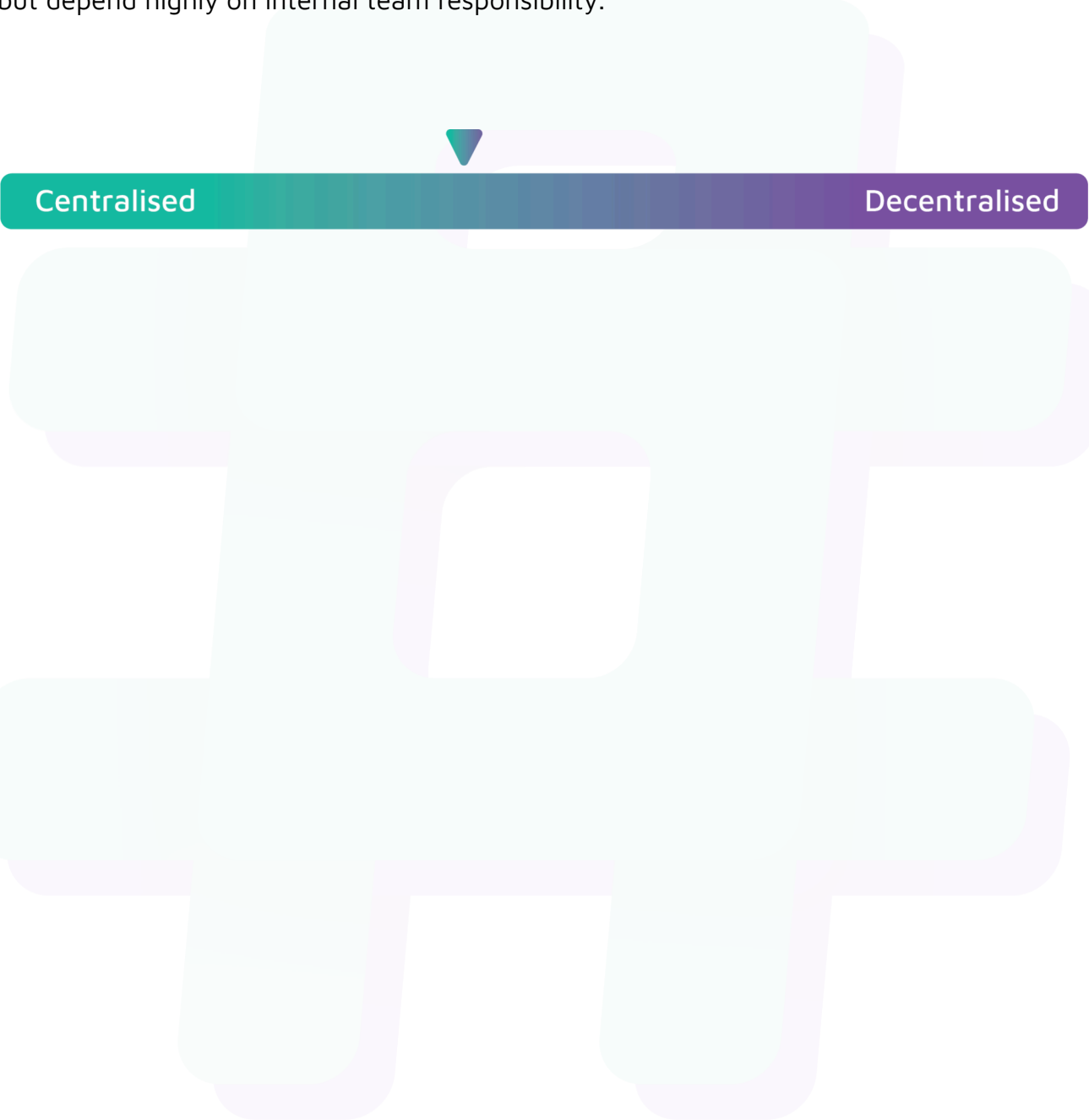
Resolved



Centralisation

The Shezmu project values security and utility over decentralisation.

The owner executable functions within the protocol increases security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Shezmu project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.